UNIVERSIDAD POLITÉCNICA SALESIANA ECUADOR

POSGRADOS

Maestría en

SOFTWARE

Asignatura:

Patrones de Diseño de Software

Actividad #: 01

Patrones creacionales

Docente:

Mauricio Ortiz Ochoa

Elaborado por (Grupo 02):

Espinoza Adrian
Jara Pablo
Lara Miguel
Torres Carina
Uyaguari Sebastián

Fecha de entrega:

2026/09/17

Repositorio de código:

https://github.com/miguesj-hub/patrones-diseno-actividad-01

Introducción

El presente informe documenta la resolución de la Actividad de aprendizaje 01 de la asignatura Patrones de Diseño de Software, orientada al resultado de aprendizaje RA1: seleccionar y aplicar patrones creacionales para resolver problemas de creación de objetos, revisando críticamente las alternativas elaboradas con apoyo de inteligencia artificial.

Se abordan dos escenarios. En el primero, el registro de vehículos y el cálculo de su matrícula, se aplica el patrón **Factory Method**; en el segundo, el registro de activos fijos institucionales, se aplica el patrón **Builder**. Para cada escenario se presentan la identificación del problema de creación de objetos, la selección y justificación del patrón, sus participantes, el diagrama de clases UML, la implementación en Java con su correspondencia con el diagrama, y el análisis del uso crítico de inteligencia artificial. El informe concluye con las conclusiones del grupo y las referencias consultadas.

El código fuente de ambas soluciones se encuentra en el repositorio del grupo: https://github.com/miguesj-hub/patrones-diseno-actividad-01.

Escenario 01: Registro de vehículos y cálculo de matrícula (Factory Method)

Justificación

Identificación del problema de creación de objetos

La empresa requiere registrar vehículos y calcular el valor de su matrícula para tres tipos concretos: Auto, Camioneta y Camión. Todos comparten un conjunto de datos (placa, marca, modelo, año de fabricación y avalúo) y una operación común para obtener el costo de matrícula; sin embargo, cada tipo aplica una regla de cálculo distinta que puede depender de atributos propios, como la capacidad y el uso en la camioneta o el tonelaje en el camión.

El análisis del enunciado permite identificar que el problema central no es el cálculo en sí, sino **la creación del objeto**: el tipo concreto de vehículo se conoce únicamente *en tiempo de ejecución*, a partir de la información del cliente, y la empresa prevé incorporar nuevas variantes (motocicletas, buses o vehículos eléctricos). Si la lógica principal instanciara directamente `new Auto(...)`, `new Camioneta(...)` o `new Camion(...)`, quedaría acoplada a las clases concretas y cada nuevo tipo obligaría a modificar el código existente, en contra del principio abierto/cerrado (Meyer, 1997; Martin, 2017).

Requisitos identificados

Requisitos funcionales identificados en el enunciado:

- Registrar vehículos de tipo Auto, Camioneta y Camión.
- Almacenar placa, marca, modelo, año de fabricación y avalúo.
- Permitir una operación para calcular el valor de la matrícula.
- Aplicar una lógica de cálculo específica según el tipo de vehículo; el enunciado ofrece ejemplos de cómo varía el cálculo en cada tipo.
- Permitir seleccionar el tipo de vehículo durante la ejecución.
- Permitir que el proceso trabaje con un contrato común de vehículo y no con el tipo concreto.

Requisitos de diseño:

- Evitar que la lógica principal dependa de las clases concretas Auto, Camioneta y Camión.
- Centralizar las responsabilidades de creación en componentes especializados.
- Facilitar la incorporación de nuevos tipos de vehículos.
- Favorecer el polimorfismo para operar con diferentes vehículos.

Clases identificadas

Las entidades identificadas a partir del enunciado son:

- **Vehículo:** concepto general que agrupa los datos comunes y la operación de cálculo de matrícula.
- **Auto**, **Camioneta** y **Camión:** tipos concretos, cada uno con su propia regla de cálculo.
- **Cliente:** parte de la aplicación que, a partir de la información proporcionada, decide en tiempo de ejecución qué tipo de vehículo registrar.

Selección y justificación del patrón

Se selecciona el patrón **Factory Method**, cuyo propósito es “definir una interfaz para crear un objeto, pero dejar que sean las subclases quienes decidan qué clase instanciar” (Gamma et al., 1994). En este escenario, la clase creadora `CreadorVehiculo` declara el método de fabricación `crearVehiculo(...)` y cada creador concreto decide qué vehículo instanciar. De este modo, una vez elegido el creador, el resto del proceso trabaja exclusivamente con las abstracciones `CreadorVehiculo` y `Vehiculo`, lo que

satisface los requisitos de desacoplamiento y extensibilidad (Freeman y Robson, 2020; Ortiz Ochoa, 2026). Asimismo, asignar la responsabilidad de creación a clases especializadas es coherente con los principios GRASP de *Creador*, *Alta cohesión* y *Variaciones protegidas* (Larman, 2004).

Durante el análisis se contrastaron dos alternativas. Una *Simple Factory* con una estructura `switch` resolvería la creación en un único punto, pero obligaría a modificar esa clase cada vez que se agregue un tipo de vehículo; además, no figura en el catálogo de Gamma et al. (1994) como patrón, sino como una práctica de programación (Freeman y Robson, 2020). Por otra parte, *Abstract Factory* está orientado a crear **familias** de productos relacionados, situación que no se presenta en el enunciado, donde existe una sola jerarquía de productos (Shvets, s.f.-b). Por ello, Factory Method es la alternativa con menor complejidad que cumple todos los requisitos.

Ventajas	Limitaciones
Evita crear directamente los vehículos en la lógica principal.	Es necesario crear clases adicionales.
Permite manejar los vehículos mediante una clase o interfaz común.	El diseño inicial es un poco más extenso.
Facilita agregar nuevos tipos de vehículos (principio abierto/cerrado).	Se necesita un creador específico para cada tipo de vehículo.
Organiza la responsabilidad de crear los objetos.	Hay que decidir cómo seleccionar el creador adecuado.

Participantes del patrón

Participante	Clase	Responsabilidad
Product	`Vehiculo` (abstracta)	Define los atributos comunes y el contrato `calcularMatricula()`.
ConcreteProduct	`Auto`, `Camioneta`, `Camion`	Implementan su propia regla de cálculo de matrícula.
Creator	`CreadorVehiculo` (abstracta)	Declara el método de fabricación `crearVehiculo(...)`, que retorna un `Vehiculo`.
ConcreteCreator	`CreadorAuto`, `CreadorCamioneta`, `CreadorCamion`	Sobrescriben `crearVehiculo(...)` e instancian el producto concreto.
Cliente	`Main`	Selecciona el creador en tiempo de ejecución y trabaja solo con las abstracciones.

Diagrama de clases

El diagrama se elaboró en dos iteraciones. La primera versión (Figura 1) se obtuvo a partir del análisis del grupo y de la revisión con IA. La versión final (Figura 2) incorpora la firma de `crearVehiculo(...)` con los atributos comunes y los constructores de los creadores concretos que reciben los datos propios de cada tipo. Se utiliza generalización para las jerarquías de productos y creadores, y dependencias estereotipadas `<<create>>` desde cada creador concreto hacia el producto que instancia, conforme a la notación de UML 2.5.1 (Object Management Group, 2017; Fowler, 2003).

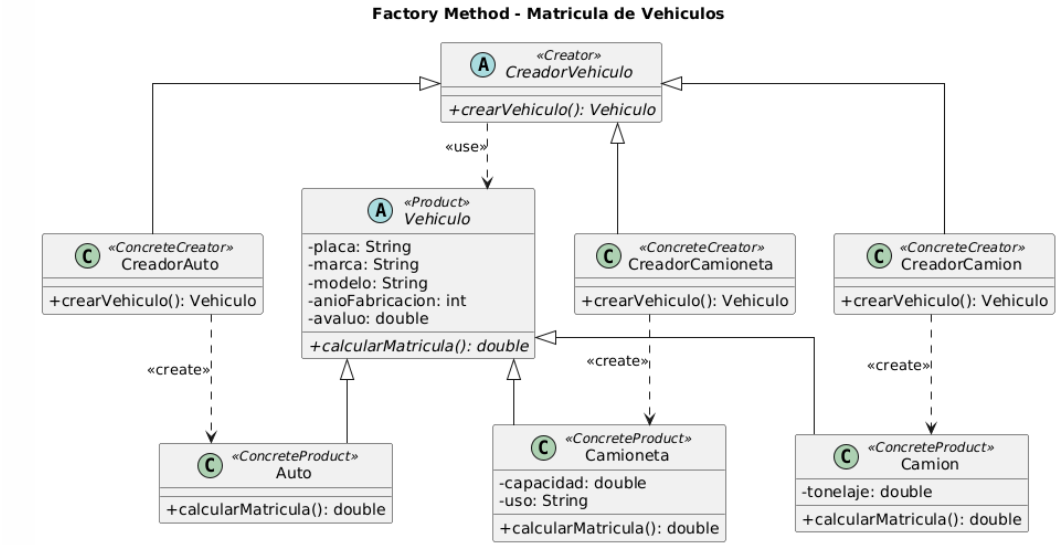


Figura 1. Diagrama de clases inicial de Factory Method.

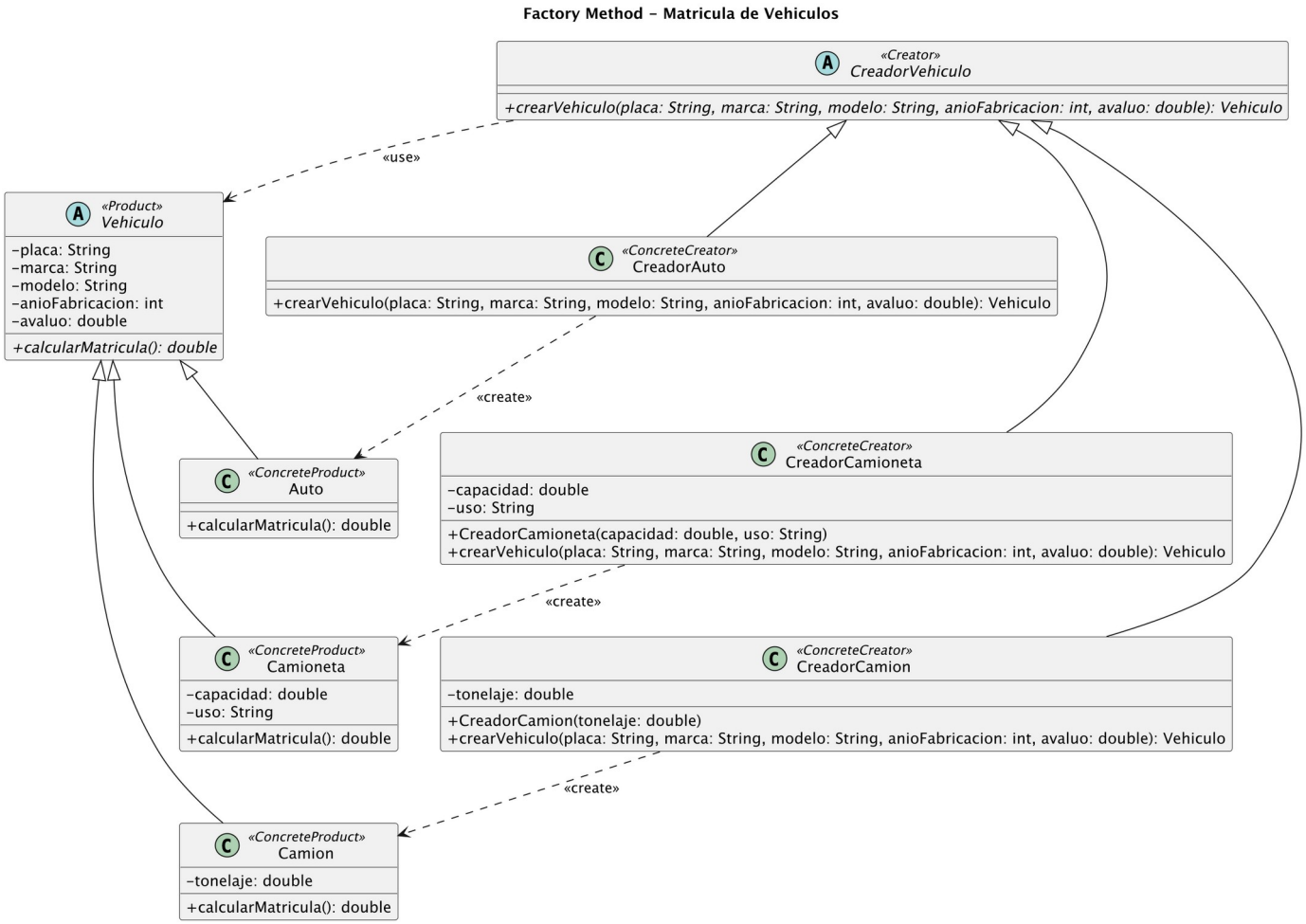


Figura 2. Diagrama de clases final de Factory Method (fuente: `diagrama.tex`, PlantUML).

Código fuente

- **Repositorio:** https://github.com/miguesj-hub/patrones-diseno-actividad-01
- **Ubicación:** carpeta `codigo-escenario-1/`, paquete `vehiculos` (Java 17).
- **Ejecución:** `javac -d out $(find src -name "*.java")` y luego `java -cp out vehiculos.Main`.

La correspondencia entre el diagrama UML y el código es directa:

Participante	Clase en el código
Product	`vehiculos.producto.Vehiculo`
ConcreteProduct	`vehiculos.producto.Auto`, `Camioneta`, `Camion`
Creator	`vehiculos.creador.CreadorVehiculo`
ConcreteCreator	`vehiculos.creador.CreadorAuto`, `CreadorCamioneta`, `CreadorCamion`
Cliente	`vehiculos.Main`

El siguiente extracto muestra el método de fabricación y el punto del cliente que trabaja únicamente con las abstracciones:

```
public abstract class CreadorVehiculo {
    public abstract Vehiculo crearVehiculo(String placa, String marca, String modelo,
                                           int anioFabricacion, double avaluo);
}

public class CreadorCamion extends CreadorVehiculo {
    private final double tonelaje;
    public CreadorCamion(double tonelaje) { this.tonelaje = tonelaje; }
    @Override
    public Vehiculo crearVehiculo(String placa, String marca, String modelo,
                                  int anioFabricacion, double avaluo) {
        return new Camion(placa, marca, modelo, anioFabricacion, avaluo, tonelaje);
    }
}

// Main.registrar(): no depende de Auto, Camioneta ni Camion
Vehiculo vehiculo = creador.crearVehiculo(placa, marca, modelo, anio, avaluo);
System.out.printf("Valor de matrícula: $%.2f%n", vehiculo.calcularMatricula());
```

Las reglas de cálculo no están definidas en el enunciado; por ello, el grupo asumió fórmulas con fines académicos: **Auto** = 2 % del avalúo + 1 % por año de antigüedad (máximo 10 %); **Camioneta** = 3 % del avalúo + 20 por tonelada de capacidad, con un recargo de 15 % si el uso es comercial; **Camión** = 4 % del avalúo + 50 por tonelada. A continuación se muestra una ejecución de prueba registrando una camioneta de uso comercial:

```
=== Registro de vehículos y cálculo de matrícula ===
Tipo de vehículo: 1) Auto  2) Camioneta  3) Camión
Opción: 2   Capacidad (t): 1.5   Uso: comercial
Placa: ABC-123   Marca: Toyota   Modelo: Hilux   Año: 2020   Avalúo: 30000


Vehículo registrado: Camioneta [placa=ABC-123, marca=Toyota, modelo=Hilux,
  año=2020, avalúo=30000,00] capacidad=1,50 t, uso=comercial
Valor de matrícula: $1069,50
```

Para incorporar un nuevo tipo, por ejemplo `Motocicleta`, basta con crear la subclase `Motocicleta extends Vehiculo` y su creador `CreadorMotocicleta extends CreadorVehiculo`, y registrar la opción en `Main.seleccionarCreador()`; el método `registrar()` y las clases existentes no se modifican.

Ajustes realizados durante la implementación:

Cambio	Justificación
Se agregaron `validarNoNegativo()` y `antiguedad()` como métodos protegidos en `Vehiculo`.	Evitan repetir validaciones y el cálculo de antigüedad en cada subclase. No alteran el contrato público del diagrama.
Se agregaron validaciones en los constructores (placa obligatoria, año de fabricación ≥ 1900, valores no negativos).	El enunciado exige atributos válidos para calcular la matrícula; así no se crean vehículos inconsistentes.
Los atributos `capacidad`, `uso` y `tonelaje` se declararon `final`.	Un vehículo no debe cambiar sus datos tras su creación.
Se agregaron *getters* y `toString()`.	Permiten mostrar el vehículo creado en la consola.
Se definieron fórmulas de matrícula académicas: Auto = 2 % del avalúo + 1 % del avalúo por año de antigüedad (recargo máximo de 10 %); Camioneta = 3 % del avalúo + 20 por tonelada de capacidad, con +15 % sobre el total si el uso es comercial; Camión = 4 % del avalúo + 50 por tonelada.	El enunciado solo da criterios generales, por lo que son un **supuesto** del grupo.
Los datos propios de cada tipo (`capacidad`, `uso`, `tonelaje`) se reciben en el constructor del creador concreto.	Conserva la misma firma de `crearVehiculo(...)` en todos los creadores, condición necesaria para el polimorfismo.

Cambio	Justificación
Se agregó `Main.java` con un menú que selecciona el creador según el tipo elegido y solicita sus datos.	Demuestra que, tras elegir el creador, el resto del proceso solo usa `CreadorVehiculo` y `Vehiculo`.

Uso crítico de inteligencia artificial

- **Prompt principal:**

  Se utilizó ChatGPT como asistente para revisar el análisis previo del grupo y elaborar una propuesta de diagrama. Conversación completa: https://chatgpt.com/share/6ac9002d-c300-83e8-9d1d-4868c9909a5b.

> Actúa como un asistente especializado en patrones de diseño de software y modelado UML.
> Estoy desarrollando una actividad académica sobre patrones creacionales. Para el primer escenario he seleccionado el patrón **Factory Method** y ya realicé un análisis previo del problema.
> **Objetivo:** Necesito que revises mi análisis y me ayudes a elaborar una propuesta de diagrama de clases UML utilizando Factory Method.
> **Instrucciones:**
> 1. Revisa el enunciado original y compáralo con mi análisis para identificar posibles errores, requisitos faltantes o aspectos que deban aclararse.
> 2. Evalúa si la selección de Factory Method es adecuada para resolver el problema e indica brevemente por qué.
> 3. Propón las clases necesarias para aplicar el patrón, utilizando su estructura tradicional: Product, ConcreteProduct, Creator y ConcreteCreator.
> 4. Identifica los atributos y métodos principales de cada clase, considerando únicamente los requisitos del escenario.
> 5. Explica las relaciones que deben existir entre las clases y el tipo de relación UML que corresponde.
> 6. Genera una propuesta de diagrama de clases UML utilizando PlantUML.
> 7. Justifica brevemente las principales decisiones del diseño.
> 8. Identifica posibles limitaciones o aspectos que podrían mejorarse.
> **Consideraciones:**
> • Utiliza un lenguaje sencillo, ya que estamos aprendiendo patrones de diseño.
> • No agregues clases o funcionalidades innecesarias.
> • Respeta la estructura de Factory Method explicada en el material académico.
> • No inventes requisitos que no estén presentes en el enunciado. Si necesitas asumir algo, indícalo claramente.
> • No generes código de implementación todavía.
> • Considera el diagrama como una propuesta que debe ser revisada antes de su implementación.
> **Enunciado original:** [se adjuntó el enunciado completo del Escenario 01]
> **Mi análisis previo:** [se adjuntó el análisis previo del grupo: problema, requisitos y clases identificadas]

Recomendaciones de la IA:

Apartado	Decisión
Recomendación aceptada	Utilizar `Vehiculo` como clase abstracta. Recomendada por la IA y también propuesta en el análisis inicial del grupo.
Recomendación modificada	Cambiar el método `crearVehiculo()` para que reciba los atributos comunes.

En general, la explicación de la IA confirmó que el diagrama, las clases inferidas y las relaciones del Factory Method eran correctas; aun así, cada recomendación se evaluó frente al enunciado antes de incorporarla.

- **Recomendación aceptada (con justificación):** definir `Vehiculo` como **clase abstracta**.

  Se acepta que `Vehiculo` sea una clase abstracta porque todos los vehículos comparten placa, marca, modelo, año de fabricación y avalúo, y cada tipo debe implementar su propia lógica de `calcularMatricula()` bajo un contrato común; así, la lógica principal trabaja con `Vehiculo` sin depender de Auto, Camioneta o Camión. A diferencia de una interfaz, la clase abstracta permite reutilizar ese estado común y su validación. Además, la recomendación coincidía con lo planteado en el análisis inicial del grupo.

- **Recomendación modificada (con justificación):** la firma del método `crearVehiculo()`.

  La IA propuso un método de fabricación sin parámetros. El grupo consideró necesario que reciba los atributos comunes (`placa`, `marca`, `modelo`, `anioFabricacion` y `avaluo`) para que cada vehículo pueda crearse con su información correspondiente. La modificación facilita el registro de los vehículos y mantiene la estructura del patrón Factory Method. Los atributos propios de cada tipo se trasladaron al constructor del creador concreto, de modo que todos los creadores conservan la misma firma y el cliente puede invocarlos de forma polimórfica.

Escenario 02: Registro de activos fijos (Builder)

Justificación

Identificación del problema de creación de objetos

La institución necesita registrar activos fijos de naturaleza diversa (computadores, monitores, impresoras, muebles, vehículos y otros bienes). Todo activo exige tres datos obligatorios —código institucional, nombre o descripción y precio de adquisición— y admite hasta diez datos opcionales cuya presencia depende del tipo de bien: un computador requiere marca, modelo, número de serie, procesador y garantía, mientras que una mesa solo necesita material, dimensiones, ubicación y custodio.

El problema de creación surge por la **combinación de atributos obligatorios y opcionales**. Resolverlo con constructores sobrecargados conduce al antipatrón del *constructor telescópico*, en el que llamadas como `new ActivoFijo("AF001", "Computador", 1200, "Dell", "Latitude", null, ...)` resultan difíciles de leer, propensas a errores de orden de parámetros y obligan a pasar valores `null` para datos que no aplican. La alternativa de constructor vacío con métodos *set* tampoco es adecuada, porque permite que el objeto exista en un estado inconsistente durante su configuración (Bloch, 2018).

Requisitos identificados

Requisitos funcionales identificados en el enunciado:

- Registrar diferentes tipos de activos institucionales.
- Registrar obligatoriamente el código institucional, el nombre o descripción y el precio de adquisición.
- Registrar los atributos opcionales según el tipo de activo y construir los activos progresivamente.

Requisitos de diseño:

- Evitar el uso de múltiples constructores con diferentes combinaciones de parámetros.
- Evitar tener que proporcionar valores vacíos o `null` para atributos que no corresponden al activo.
- Mantener un proceso de construcción claro y fácil de comprender, que entregue un objeto correctamente configurado.
- Separar la construcción progresiva del activo de su representación final.

Clases identificadas

Las entidades identificadas a partir del enunciado son: **Activo Fijo**, **Computador**, **Monitor**, **Impresora**, **Mueble** y **Vehículo**. Durante el diseño se determinó que los tipos de activo no presentan comportamiento propio, sino únicamente datos distintos; por ello, se representan mediante el atributo `tipoActivo` de una sola clase `ActivoFijo` y no como subclases (véase la recomendación aceptada en el uso crítico de IA).

Atributos identificados

Atributos obligatorios	Atributos opcionales
Código institucional	Marca · Modelo · Número de serie
Nombre o descripción	Ubicación · Custodio responsable · Fecha de adquisición
Precio de adquisición	Período de garantía · Proveedor · Características técnicas · Observaciones

Selección y justificación del patrón

Se selecciona el patrón **Builder**, cuyo propósito es "separar la construcción de un objeto complejo de su representación, de modo que el mismo proceso de construcción pueda crear diferentes representaciones" (Gamma et al., 1994). Bloch (2018) lo recomienda expresamente cuando una clase presenta muchos parámetros, en particular si varios de ellos son opcionales, porque combina la seguridad del constructor telescópico con la legibilidad de los métodos *set*. En este escenario, `ConstructorActivoFijo` acumula los datos paso a paso y solo entrega el `ActivoFijo` en `obtenerActivo()`, tras validar los obligatorios, mientras que `DirectorActivoFijo` encapsula la secuencia común de construcción de la base obligatoria (Ortiz Ochoa, 2026; Shvets, s.f.-a).

Se descartó *Factory Method* porque el problema no radica en decidir **qué clase** instanciar, sino en **cómo** configurar un único tipo de objeto con numerosas combinaciones de atributos. Asimismo, se descartó una jerarquía de productos por tipo de activo, ya que el enunciado no asigna comportamiento diferente a cada bien, sino solo datos distintos.

Ventajas	Limitaciones
Permite construir los activos fijos paso a paso.	Es necesario crear clases adicionales para manejar la construcción.
Facilita el manejo de atributos obligatorios y opcionales.	El diseño inicial puede ser un poco más extenso.
Evita utilizar constructores con demasiados parámetros.	Se debe controlar que los atributos obligatorios estén presentes.
Permite crear activos con diferentes configuraciones.	Puede resultar innecesario para objetos que tienen pocos atributos.

Ventajas	Limitaciones
Hace que la creación de los objetos sea más clara y organizada.	Requiere definir los pasos o métodos para construir el objeto.

Participantes del patrón

Participante	Clase	Responsabilidad
Product	`ActivoFijo`	Representa el activo final; es inmutable y solo puede crearlo el builder.
Builder	`ActivoFijoBuilder` (interfaz)	Declara los pasos de construcción y la operación `obtenerActivo()`.
ConcreteBuilder	`ConstructorActivoFijo`	Mantiene los datos en construcción, valida los obligatorios y crea el producto.
Director	`DirectorActivoFijo`	Inicia la construcción y establece la base obligatoria (`construirBase`).
Cliente	`Main`	Usa el Director para la base y agrega en el builder los opcionales de cada activo.

Diagrama de clases

La Figura 3 corresponde a la propuesta inicial y la Figura 4 al diagrama final, ajustado durante la implementación. Los cambios principales fueron: la relación entre `ConstructorActivoFijo` y `ActivoFijo` pasó de asociación a **dependencia** `<<create>>`, porque el builder no conserva el producto como atributo; se listaron los campos temporales del ConcreteBuilder; se agregaron multiplicidades; y se emplearon los tipos `BigDecimal` y `LocalDate` para valores monetarios y fechas.

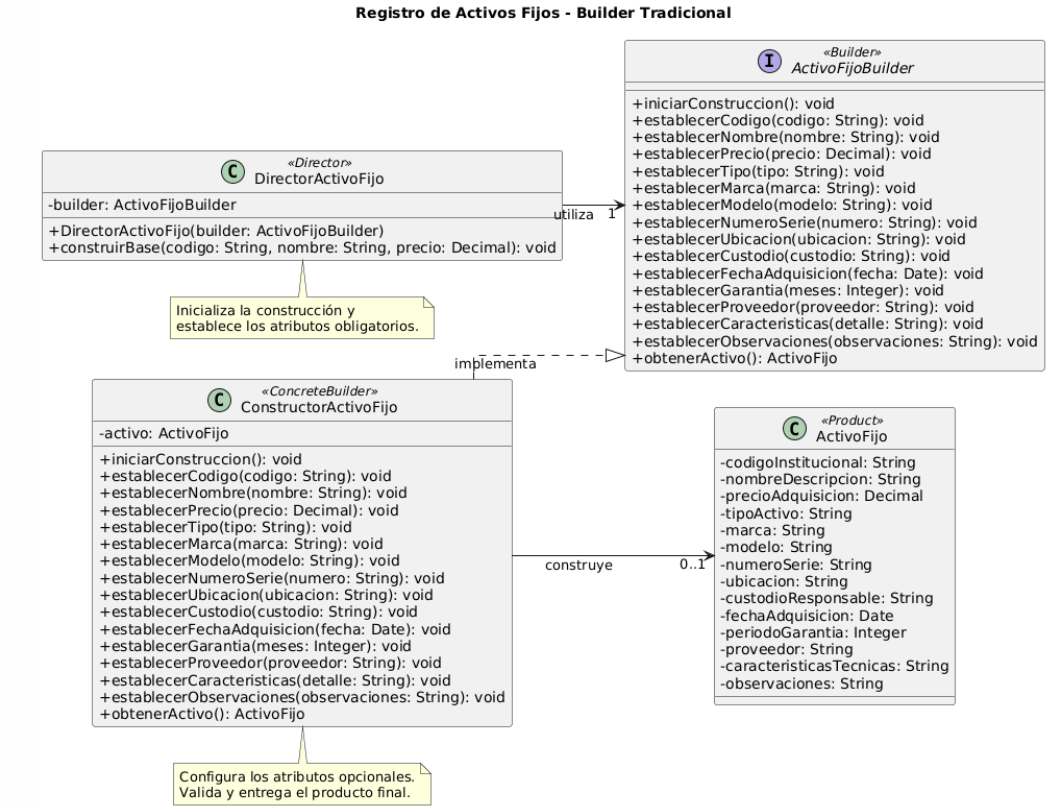


Figura 3. Diagrama de clases inicial de Builder.



Figura 4. Diagrama de clases final de Builder (fuente: `diagrama.tex`, PlantUML).

Código fuente

- **Repositorio:** https://github.com/miguesj-hub/patrones-diseno-actividad-01
- **Ubicación:** carpeta `codigo-escenario-2/`, paquete `activos` (Java 17).
- **Ejecución:** `javac -d out $(find src -name "*.java")` y luego `java -cp out activos.Main`.

La correspondencia entre el diagrama UML y el código es la siguiente:

Participante	Clase en el código
Product	`activos.construccion.ActivoFijo`
Builder	`activos.construccion.ActivoFijoBuilder`
ConcreteBuilder	`activos.construccion.ConstructorActivoFijo`
Director	`activos.construccion.DirectorActivoFijo`
Cliente	`activos.Main`

El extracto muestra la construcción progresiva de un computador: el Director fija los obligatorios y el cliente agrega solo los opcionales que corresponden; `obtenerActivo()` valida y entrega el producto:

```
director.construirBase("AF001", "Computador portátil", new BigDecimal("1200.00"));
builder.establecerTipo("Computador");
builder.establecerMarca("Dell");
builder.establecerModelo("Latitude 5440");
builder.establecerNumeroSerie("DL-5440-88213");
builder.establecerCaracteristica("Procesador", "Intel Core i7");
builder.establecerGarantia(36);
ActivoFijo computador = builder.obtenerActivo();


// ConstructorActivoFijo.obtenerActivo()
if (esVacio(codigo)) throw new IllegalStateException("Falta el código institucional");
if (esVacio(nombre)) throw new IllegalStateException("Falta el nombre o descripción");
if (precio == null || precio.signum() <= 0)
    throw new IllegalStateException("Falta el precio de adquisición o no es mayor que cero");
ActivoFijo resultado = new ActivoFijo(codigo, nombre, precio, tipo, marca, ...);
iniciarConstruccion();   // reinicia el builder para el siguiente activo
return resultado;
```

Resultado de la ejecución: dos activos con configuraciones distintas y un tercero rechazado por carecer de datos obligatorios.

```
=== Registro de activos fijos ===
ActivoFijo
  código: AF001 | nombre: Computador portátil | precio: 1200.00 | tipo: Computador
  marca: Dell | modelo: Latitude 5440 | número de serie: DL-5440-88213
  fecha de adquisición: 2026-03-15 | garantía (meses): 36 | proveedor: Tecnología Andina S.A.
  características: {Procesador=Intel Core i7, Memoria RAM=16 GB, Almacenamiento=512 GB SSD}
ActivoFijo
  código: AF002 | nombre: Mesa de reuniones | precio: 350.00 | tipo: Mueble
  ubicación: Cuenca - Sala de juntas 2 | custodio: Departamento Administrativo
  características: {Material=Madera de roble, Dimensiones=2,40 x 1,10 m}
Activo rechazado: Falta el nombre o descripción
```

Ajustes realizados durante la implementación:

Cambio	Justificación
`ActivoFijo` se implementó como clase **inmutable** (campos `final`, constructor de paquete, sin *setters*); el `Map` de características se copia y se hace no modificable.	Garantiza que un activo, una vez construido y validado, no pueda quedar inválido ni modificarse por accidente.
`ConstructorActivoFijo` guarda los datos en campos temporales en lugar del atributo `activo: ActivoFijo`.	Consecuencia de la inmutabilidad: el producto solo puede crearse cuando todos los datos están completos.
La relación `ConstructorActivoFijo --> ActivoFijo : construye` se cambió por una **dependencia** `ConstructorActivoFijo ..> ActivoFijo : <<create>>`.	El builder no conserva al producto como atributo; solo lo crea en `obtenerActivo()`. Una asociación implicaría lo contrario.
Se agregaron las multiplicidades * → 1 entre `DirectorActivoFijo` y `ActivoFijoBuilder`, y entre `ConstructorActivoFijo` y `ActivoFijo`.	Alinea el diagrama con la estructura del patrón Builder del material de la asignatura (Unidad 01, sección 03), donde la flecha punteada `<<create>>` va de ConcreteBuilder a Producto.
En el diagrama de `ConstructorActivoFijo` se listaron los campos temporales y se actualizó su nota.	El ConcreteBuilder mantiene la representación que se está construyendo, valida los obligatorios y crea el producto en `obtenerActivo()`.
`Decimal` se reemplazó por `BigDecimal` y `Date` por `LocalDate`.	Son los tipos de Java adecuados para valores monetarios y fechas.
`obtenerActivo()` valida los atributos obligatorios (código, nombre y precio mayor que cero) y lanza `IllegalStateException` si faltan.	Cumple el requisito de entregar un objeto “correctamente configurado” y controla los obligatorios, limitación identificada en el análisis.
`obtenerActivo()` reinicia el builder al terminar.	Permite construir otro activo con el mismo builder sin arrastrar datos del anterior.

Cambio	Justificación
`establecerGarantia` rechaza valores negativos y `establecerCaracteristica` rechaza claves vacías.	Evita datos opcionales incoherentes.
`DirectorActivoFijo.construirBase()` invoca `iniciarConstruccion()` antes de fijar los obligatorios, y el constructor del Director valida que el builder no sea nulo.	Asegura que cada construcción parta de un estado limpio.
Los métodos del builder se mantienen como `void` (builder clásico).	Coincide con el diagrama y con la participación del Director; el estilo encadenado (variante moderna de Bloch, 2018) haría innecesario al Director.
Se agregó `Main.java`, que usa el Director para la base y el builder directamente para los opcionales y las características técnicas.	Demuestra la construcción paso a paso que pide el enunciado.

Uso crítico de inteligencia artificial

- **Prompt principal:**

  Conversación completa: https://chatgpt.com/share/6ac90712-ac80-83e9-a6c9-04948b7a77bd.

> Actúa como un asistente especializado en patrones de diseño de software y modelado UML.
> Estoy desarrollando una actividad académica sobre patrones creacionales. Para el segundo escenario he seleccionado el patrón **Builder** y ya realicé un análisis previo del problema.
> **Objetivo:** Necesito que revises mi análisis y me ayudes a elaborar una propuesta de diagrama de clases UML utilizando el patrón Builder en su estructura tradicional.
> **Instrucciones:**
> 1. Revisa el enunciado original y compáralo con mi análisis para identificar posibles errores, requisitos faltantes o aspectos que deban aclararse.
> 2. Evalúa si la selección de Builder es adecuada para resolver el problema e indica brevemente por qué.
> 3. Propón las clases necesarias para aplicar el patrón, utilizando su estructura tradicional: Product, Builder, ConcreteBuilder y Director.
> 4. Identifica los atributos y métodos principales de cada clase, considerando únicamente los requisitos del escenario.
> 5. Explica cómo se manejarán los atributos obligatorios y opcionales durante la construcción de los activos fijos.
> 6. Explica las relaciones que deben existir entre las clases y el tipo de relación UML que corresponde.
> 7. Genera una propuesta de diagrama de clases UML utilizando PlantUML.
> 8. Justifica brevemente las principales decisiones del diseño.
> 9. Identifica posibles limitaciones o aspectos que podrían mejorarse.
> **Consideraciones:**
> • Utiliza un lenguaje sencillo, ya que estamos aprendiendo patrones de diseño.
> • No agregues clases o funcionalidades innecesarias.
> • Respeta la estructura tradicional de Builder explicada en el material académico.
> • Considera inicialmente las clases ActivoFijo (Product), ActivoFijoBuilder (Builder), ConstructorActivoFijo (ConcreteBuilder) y DirectorActivoFijo (Director), pero evalúa si sus responsabilidades son adecuadas.
> • No crees una clase independiente para cada tipo de activo fijo, salvo que exista una justificación basada en el enunciado.
> • No inventes requisitos que no estén presentes en el escenario. Si necesitas asumir algo, indícalo claramente.
> • No generes código de implementación todavía.
> • Considera el diagrama como una propuesta inicial que debe ser revisada antes de implementarse.
> **Enunciado original:** [se adjuntó el enunciado completo del Escenario 02]
> **Mi análisis previo:** [se adjuntó el análisis previo del grupo: problema, requisitos, clases y atributos identificados]

Recomendaciones de la IA:

Apartado	Decisión
Recomendación aceptada	Utilizar una única clase producto (`ActivoFijo`), sin crear una jerarquía de productos para computadoras, impresoras, muebles y vehículos.
Recomendación modificada	Modificar el atributo `caracteristicasTecnicas` a `Map<String, String>` para almacenar diferentes características.
Recomendación rechazada	Mantener la asociación `ConstructorActivoFijo → ActivoFijo` de la propuesta inicial; se reemplazó por una dependencia `<<create>>`.

- **Recomendación aceptada (con justificación):** utilizar una **única clase producto** `ActivoFijo`.

  Se acepta la recomendación de utilizar únicamente la clase `ActivoFijo` para representar los diferentes bienes institucionales, sin crear clases independientes para computadores, impresoras, muebles o vehículos. Esta decisión permite manejar los distintos activos mediante una misma estructura, configurando únicamente los atributos necesarios para cada uno. En el enunciado, los tipos de activo no presentan comportamiento diferente, sino solo datos distintos; crear una subclase por tipo habría incrementado la complejidad sin aportar valor.

- **Recomendación modificada (con justificación):** el tipo del atributo `caracteristicasTecnicas`.

  Se modificó el atributo `caracteristicasTecnicas` de tipo `String` a `Map<String, String>`, cargado mediante `establecerCaracteristica(clave, valor)`, porque se considera más adecuado utilizar una estructura que permita almacenar diferentes características técnicas (procesador, material, dimensiones) de forma organizada y flexible, sin necesidad de crear atributos específicos para cada tipo de activo. El mapa se expone como colección no modificable para preservar la inmutabilidad del producto.

- **Recomendación rechazada (con justificación):** la asociación entre el ConcreteBuilder y el producto.

  Se rechazó mantener la asociación `ConstructorActivoFijo → ActivoFijo` sugerida en la propuesta inicial y se reemplazó por una dependencia `<<create>>`, ya que el ConcreteBuilder no conserva al producto como atributo, sino que lo crea al finalizar la construcción (Object Management Group, 2017; Ortiz Ochoa, 2026).

Conclusiones de la actividad

Esta actividad permitió comprender que el enunciado de un problema ofrece pistas importantes para elegir el patrón de diseño adecuado. En el Escenario 01, la necesidad de crear distintos tipos de vehículos durante la ejecución y de evitar la dependencia directa de clases concretas hizo de Factory Method la opción apropiada: el cliente trabaja con `CreadorVehiculo` y `Vehiculo`, por lo que se pueden agregar nuevos tipos sin modificar la lógica principal, en coherencia con el principio abierto/cerrado. En el Escenario 02, Builder resultó más adecuado porque existían atributos obligatorios y opcionales; el patrón permite construir el objeto paso a paso, validar los datos obligatorios y evitar constructores demasiado largos o difíciles de mantener.

La inteligencia artificial sirvió como apoyo para obtener una primera propuesta, pero varias decisiones tuvieron que revisarse y ajustarse según el enunciado y el material de la asignatura. Por ejemplo, se mantuvieron la clase abstracta `Vehiculo` y una sola clase `ActivoFijo`, mientras que se modificaron los parámetros de `crearVehiculo()` y el tipo de las características técnicas. Durante la implementación también se ajustó el UML para que coincidiera con el código. En general, ambos patrones agregan más clases, pero permiten una solución más clara, organizada y mantenible, y confirman que la IA complementa, pero no reemplaza, el criterio del diseñador.

Referencias bibliográficas

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Fowler, M. (2003). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.

Freeman, E., y Robson, E. (2020). *Head First design patterns: Building extensible and maintainable object-oriented software* (2.ª ed.). O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3.ª ed.). Prentice Hall.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Meyer, B. (1997). *Object-oriented software construction* (2.ª ed.). Prentice Hall.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), versión 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Ortiz Ochoa, M. (2026). *Unidad 01: Patrones creacionales* [Material de clase, secciones 01, 02 y 03]. Maestría en Software, Universidad Politécnica Salesiana.

Shvets, A. (s.f.-a). *Builder*. Refactoring.Guru. https://refactoring.guru/es/design-patterns/builder

Shvets, A. (s.f.-b). *Factory Method*. Refactoring.Guru. https://refactoring.guru/es/design-patterns/factory-method